UNIVERSITATEA NATIONALA DE STIINTE SI TEHNOLOGIE„POLITEHNICA” din BUCUREȘTI

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Proiect

Programarea Interfețelor pentru Baze de Date

Student: Cristea Denis-Petrisor

Grupa 434E

Prof. coordonator:

Ș.l. Dr. Ing. Pupezescu Valentin

Cuprins:

1. Cerinta proiectului
2. Tehnologii utilizate
3. Descrierea aplicatiei
4. Functionalitatea aplicatiei
5. Implementarea
6. Concluzii
7. Bibliografie

1. Cerinta proiectului

Proiectul propune dezvoltarea a două aplicații care utilizează o bază de date implementată în sistemul de gestionare MySQL, împreună cu două interfețe pentru accesarea acesteia. Interfețele pot fi create folosind tehnologii la alegere, precum JSP, Hibernate, JPA, .NET, Python sau altele. Structura bazei de date va fi definită pe baza unei liste prestabilite de tabele, cu relații și asocieri specificate între ele.

Obiectivul principal al proiectului este de a permite utilizatorilor să efectueze operațiuni de gestionare a datelor, cum ar fi vizualizarea, adăugarea, modificarea și ștergerea acestora. Interfețele vor fi proiectate pentru a oferi o prezentare clară și eficientă a datelor interconectate între diverse tabele, subliniind relațiile existente dintre acestea.

2. Tehnologii utilizate

MySQL este un sistem de gestionare a bazelor de date relaționale, apreciat pentru performanța și fiabilitatea sa în manipularea datelor. Folosit pe scară largă în proiecte de toate dimensiunile, MySQL este compatibil cu diverse limbaje de programare și sisteme de operare. Suportul său robust pentru tranzacții, securitatea avansată și comunitatea extinsă de dezvoltatori contribuie la popularitatea sa continuă în industrie. [1][5]

Spring Boot este un framework bazat pe Java care facilitează dezvoltarea aplicațiilor web printr-o configurare minimală și un set integrat de funcționalități. Spring Boot permite crearea rapidă a aplicațiilor independente, ușor de configurat și scalabile, iar integrarea sa cu diverse baze de date relaționale, cum ar fi MySQL, face ca gestionarea datelor să fie eficientă și flexibilă. Framework-ul susține dezvoltarea aplicațiilor RESTful și permite implementarea rapidă a API-urilor, oferind o experiență simplificată de dezvoltare. [2][5][6]

HTML (HyperText Markup Language) este limbajul standard utilizat pentru structurarea conținutului pe web, definind elemente esențiale precum paragrafe, titluri, link-uri și imagini. HTML formează baza oricărei pagini web și permite crearea unei structuri logice și clare a conținutului. [3]

CSS (Cascading Style Sheets) este folosit pentru a personaliza și controla aspectul vizual al elementelor HTML, inclusiv designul, culorile, fonturile și animațiile. Împreună, HTML și CSS sunt fundamentale pentru dezvoltarea paginilor web moderne și atractive, oferind o experiență vizuală consistentă și plăcută utilizatorilor. [4]

Bootstrap ste un framework front-end open-source care faciliteaza dezvoltarea rapida a aplicatiilor web, oferind sabloane de design predefinite pentru componente de interfata cu utilizatorul[7]

3. Descrierea aplicatiei

Ca temă am abordat gestionarea clientilor și distributia lor pe proiectele companiei, creând o interfață web care operează asupra bazei de date ce stochează informațiile relevante. Funcționalitățile implementate includ vizualizarea, adăugarea, modificarea și ștergerea datelor, oferind utilizatorilor o experiență intuitivă și ușor de utilizat.

Implementare

Baza de date a fost realizată utilizând **MySQL Workbench** și este compusă din trei tabele:

clienti, **proiecte** și **clienti_proiecte**. Relația dintre angajați și review-uri este de tip M:N și este gestionată prin intermediul unui tabel intermediar, numit **clienti_proiecte**.

Structura coloanelor tabelelor

TABELA CLIENTI

Această tabelă stochează informațiile despre clientii companiei, inclusiv detalii precum numele, email-ul și numarul de telefon. Structura sa este următoarea:

4. **id_angajat** – INT, cheie primară (PK), autoincrement (AI),, non-null (NN);
5. **nume** – VARCHAR(100), non-null (NN);
6. **email** – VARCHAR(100), non-null (NN),unic(UN);
7. **telefon** – VARCHAR(20);

TABELA PROIECTE

Această tabelă înregistrează proiectele companiei, precum denumirea,data de start si data de finalizare a proiectului. Structura este următoarea:

- **id_proiect** – INT, cheie primară (PK), autoincrement (AI), non-null (NN);
- **denumire**– VARCHAR(100). not null(NN);
- **data_final** – DATE
- **data_start -** DATE

- data_final-DATE.

TABELA CLIENTI_PROIECTE

Această tabelă este intermediară și gestionează asocierea dintre tabelele **clienti** și **proiecte** reprezentând relația de tip M:N. Structura este următoarea:

- **id_rol** – INT, cheie primară (PK), autoincrement (AI), non-null (NN);
- **id_client** – INT, cheie externă (FK), referință către tabela **clienti;**;
- **id_proiect** – INT, cheie externă (FK), referință către tabela **proiecte;**

- **rol_client -** VARCHAR(45);.

Relațiile între tabele

- Relația dintre **clienti** și **proiecte** este gestionată prin tabela **clienti_proiecte**:
  - Când un client este șters din tabela **clienti**, toate referințele asociate din tabela **clienti_proiecte** sunt șterse automat (ON DELETE CASCADE).
  - La ștergerea unui review din tabela **proiecte**, toate legăturile din tabela **clienti_proiecte** asociate acelui review sunt, de asemenea, eliminate.

Acest design oferă o structură logică și eficientă pentru stocarea și manipularea datelor, facilitând gestionarea informațiilor legate de angajați și review-uri într-un mod organizat și scalabil.

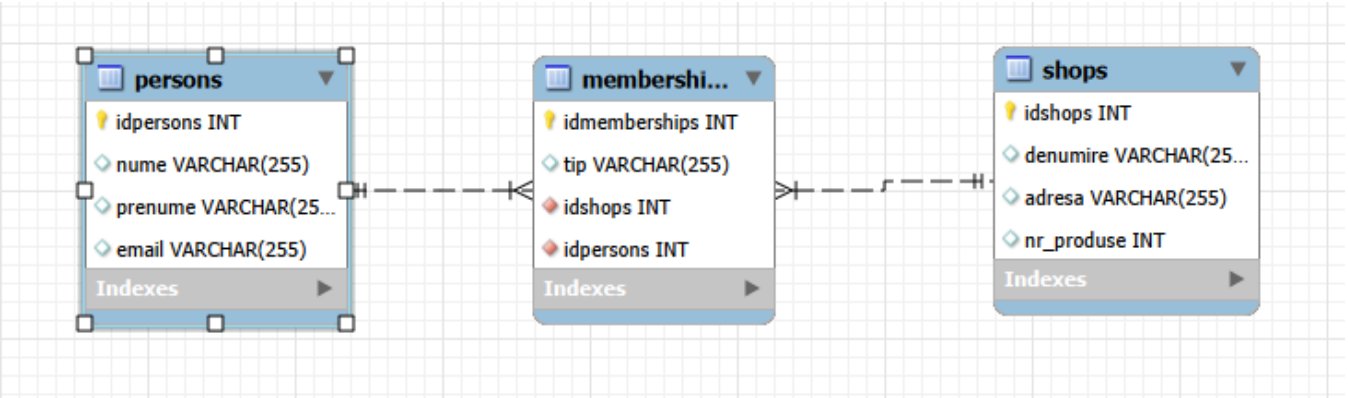

Fig. 1 – Diagrama EER a tabelelor și conținutului acestora

Ce relatii exista intre cele trei tabele?

Între **proiecte** și **clienti_proiecte** există o asociere de tip **1:N**.
Între **clienti** și **clienti_proiecte** există o asociere de tip **1:N**.
Între **clienti** și **proiecte** există o asociere de tip **M:N**, gestionată prin tabela intermediară **clienti_proiecte**.

Datorită asocierii **M:N**, **id_client** și **id_proiect** sunt setate ca chei străine (foreign keys) în tabela **clienti_proiecte**.

8. Functionalitatea aplictiei

Proiectul realizat utilizând tehnologia Spring Boot are următoarea structură:

1. Controller

- **Rol**: Controller-ul gestionează cererile HTTP primite de la client și le direcționează către logica corespunzătoare. Este responsabil pentru primirea și procesarea cererilor (GET, POST, etc.), validarea datelor și returnarea unui răspuns corespunzător (de obicei o pagină HTML sau un JSON). În Spring MVC, controller-ul este marcat cu adnotarea `@Controller` sau `@RestController` pentru API-uri RESTful.

2. Entity

- **Rol**: Entitatea reprezintă o clasă Java care corespunde unui tabel din baza de date. Aceasta este marcată cu adnotarea `@Entity` și conține câmpuri care sunt mapate pe coloanele tabelei. Entitățile sunt folosite pentru a manipula datele din baza de date prin intermediul ORM-ului (Object-Relational Mapping), în acest caz, Hibernate sau JPA (Java Persistence API).

3. Repository

- **Rol**: Repository-ul oferă un nivel de abstracție pentru interacțiunea cu baza de date. Este responsabil pentru operațiile CRUD (Create, Read, Update, Delete) și alte operațiuni complexe de interogare a bazei de date. În Spring, repository-ul este o interfață care extinde `JpaRepository` sau `CrudRepository` și permite efectuarea rapidă a interogărilor pe entitățile din baza de date fără a scrie SQL explicit.

4. HTML-uri (Template-uri)

- **Rol**: HTML-urile, de obicei, reprezintă fișiere de template procesate de un motor de template precum **Thymeleaf** sau **JSP**. Aceste fișiere definesc structura vizuală a aplicației web și permit inserarea de date dinamice în pagină (de exemplu, datele returnate de un controller). Ele sunt procesate pe server înainte de a fi trimise către client. HTML-urile sunt utilizate pentru a crea interfața utilizatorului, interacționând cu controller-ul pentru a prezenta datele sau a captura input-ul utilizatorului.

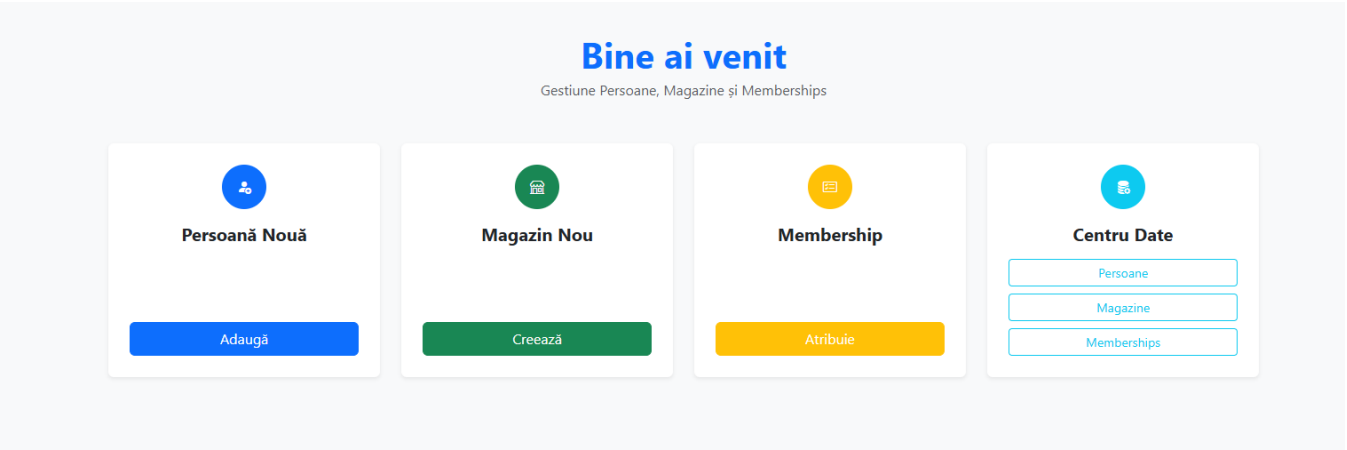

Fig. 2 – Pagina principala a interfetei

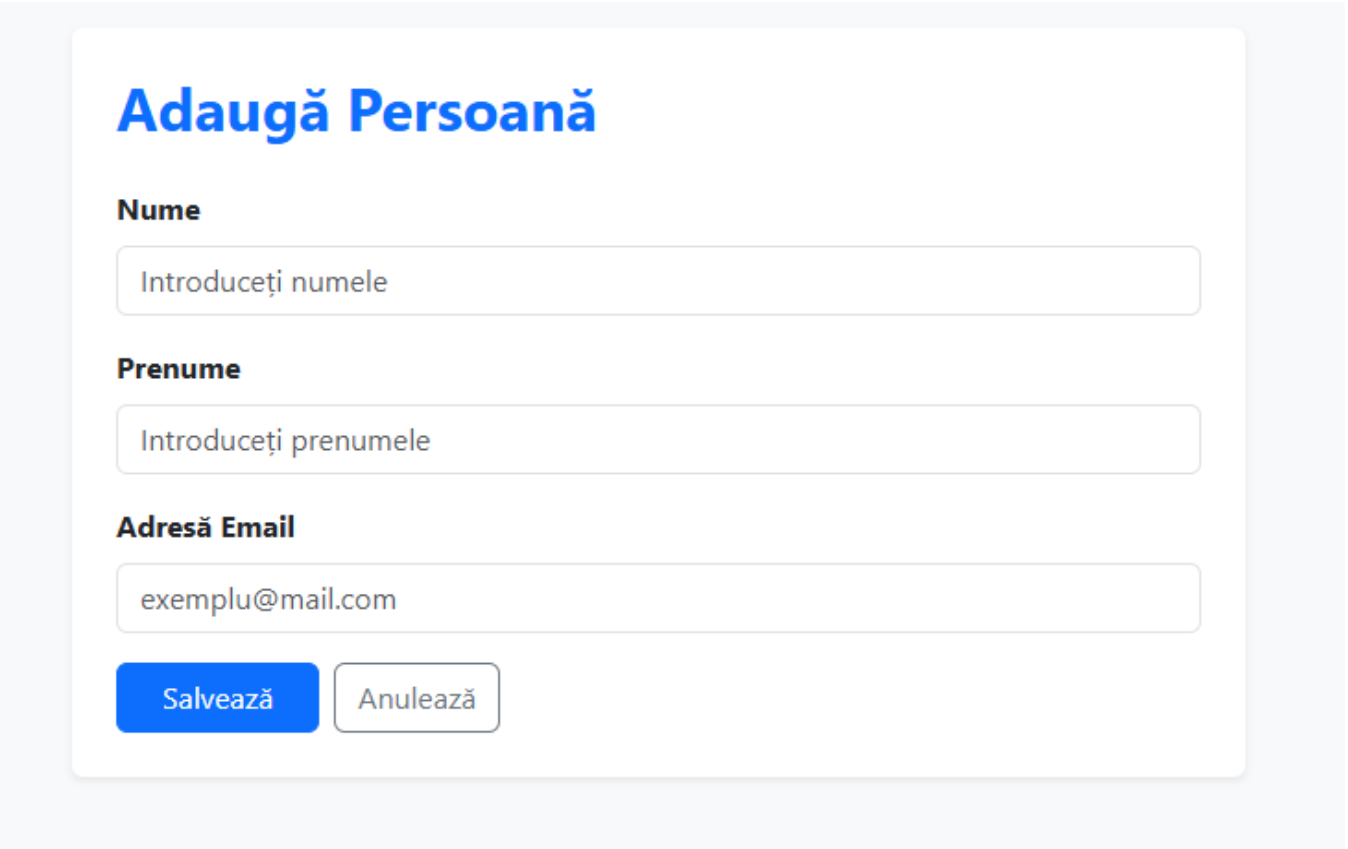

Fig. 3 – Pagina destinate adaugarii unui client

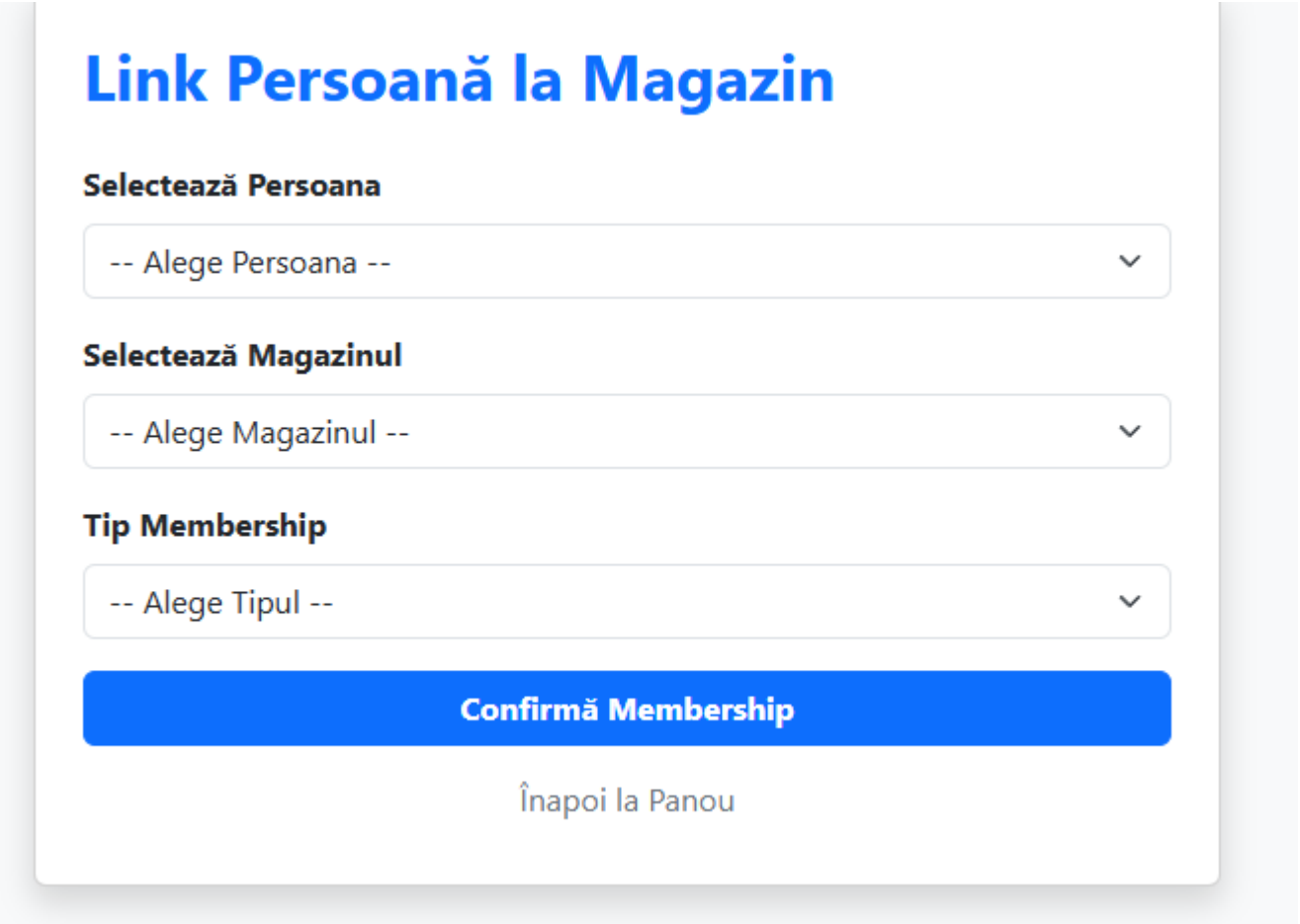

Fig. 4– Pagina destinata asocierii unui client unui magazin

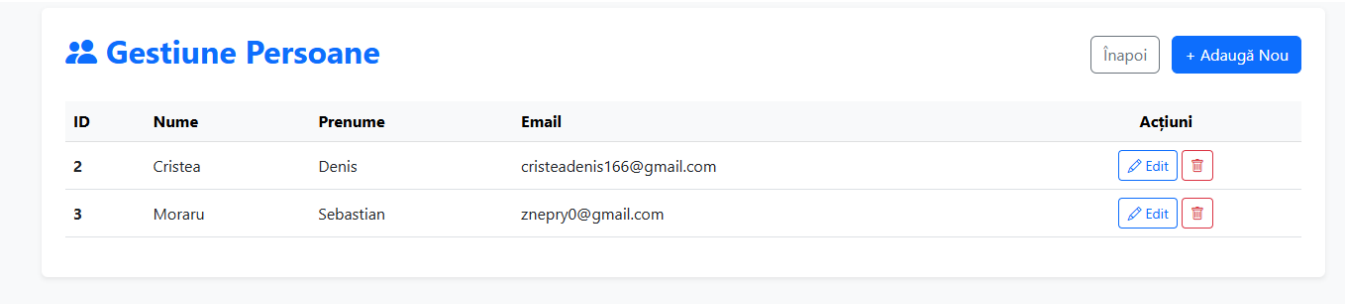

Fig. 5 -Pagina destinata vizualizarii cu setari de editare si stergere pentru clienti

9. Implementare

Se creeaza mai multe pachete java ,voi analiza in continuare cate unul din fiecare categorie:

```
package com.example.manager_pro;
```

```
import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import java.awt.Desktop;

import java.io.IOException;

import java.net.URI;

import java.net.URISyntaxException;



@SpringBootApplication

public class ManagerProApplication {



   public static void main(String[] args) {

       SpringApplication.run(ManagerProApplication.class,  args);

                                if     (Desktop.isDesktopSupported()     &&
Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)) {

           try {

                                           Desktop.getDesktop().browse(new
URI("http://localhost:8080/view/dashboard"));

           } catch (IOException | URISyntaxException e) {

                              System.out.println("Could not open browser
automatically: " + e.getMessage());

           }

       }



   }

}
```

Această clasă este punctul de intrare al aplicației Spring Boot.

- **@SpringBootApplication**: Combină configurarea aplicației și activarea auto-configurării.

- **main()**: Lansează aplicația Spring Boot folosind SpringApplication.run(), care inițializează aplicația și contextul Spring.

```
package com.example.manager_pro.Controller;


import com.example.manager_pro.clienti;

import com.example.manager_pro.Repository.clientiRepository;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;

import org.springframework.http.HttpStatus;

import org.springframework.web.server.ResponseStatusException;

import lombok.RequiredArgsConstructor;


import java.util.List;


@RestController

@RequestMapping("/api/clients") // This is the URL path

@CrossOrigin("*") // Allows your frontend to connect without security blocks

@RequiredArgsConstructor

public class clientiController {


   @Autowired

   private clientiRepository clientiRepository;


   @GetMapping("/")

   public String welcome() {

       return "<html><body><h1>CLIENTS MANAGER</h1></body></html>";

   }

   //Get All Clients
```

```
    @GetMapping

    public List<clienti> getAllClients() {

        return clientiRepository.findAll();

    }



    // Create a Client

    @PostMapping

    @ResponseStatus(HttpStatus.CREATED)

    public clienti createclient(@RequestBody clienti client) {

        return clientiRepository.save(client);

    }



    //Update a Client Data

    @PutMapping("/{id}")

    public clienti updateclient(@PathVariable Integer id, @RequestBody clienti
clientiDetails) {

        clienti client = clientiRepository.findById(id)

                                                    .orElseThrow(()    ->    new
ResponseStatusException(HttpStatus.NOT_FOUND, "Company not found"));



        client.setNume(clientiDetails.getNume());

        client.setTelefon(clientiDetails.getTelefon());

        client.setEmail(clientiDetails.getEmail());



        return clientiRepository.save(client);

    }



    // Delete a Client

    @DeleteMapping("/{id}")
```

```
    @ResponseStatus(HttpStatus.NO_CONTENT)

    public void deleteclient(@PathVariable Integer id) {

        if (!clientiRepository.existsById(id)) {

                throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Company
not found");

        }

        clientiRepository.deleteById(id);

    }

}
```

Aceasta este clasa **clientiController**, care gestionează rutele pentru operațiile CRUD (Create, Read, Update, Delete) ale entității **clientit**în cadrul aplicației Spring.

1. **@RestController**: Anunță Spring că aceasta este o clasă de tip controller pentru a manipula cererile HTTP și a returna vizualizări (pagini HTML).

2. @**RequestMapping("/api/clients")**:Anunta conectarea la un URL

3. **@RequiredArgsConstructor** permite folosirea constructorilor din libraria lombok spre simplificarea codului
4. **@GetMapping("/")**: Afișează lista clientilor prin intermediul metodei getAllClients()
5. **@ResponseStatus(HttpStatus.CREATED):**permite creearea unui nou client in tabel
6. @**PutMapping("{/id}"):** functie care permite editarea datelor unnui client
7. **@DeleteMapping("/{id}")**:functie pentru stergerea unui client din tabel
   Fiecare metodă manipulează diferite operații CRUD pentru **entitatea Angajat** folosind serviciul **AngajatService**.

```
package com.example.manager_pro;



import jakarta.persistence.*;

import lombok.AllArgsConstructor;

import lombok.Data;

import lombok.NoArgsConstructor;



@Entity

@Table(name = "clienti")

@Data
```

```
@NoArgsConstructor

@AllArgsConstructor

public class clienti {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    @Column(name = "id_client")

    private Integer id_client;


    @Column(name = "nume")

    private String nume;


    @Column(name = "email")

    private String email;


    @Column(name = "telefon")

    private  String telefon;

}
```

Aceasta este clasa **`clienti`** are reprezintă entitatea unui client în baza de date, folosită în aplicația Spring pentru gestionarea datelor legate de clienti.

1. **`@Entity`**: Marchează clasa ca entitate JPA (Java Persistence API), care va fi mapată într-o tabelă din baza de date.
2. **`@Table(name = "clienti")`**: Definește numele tabelei din baza de date asociate cu entitatea.
3. **`@Id`** și **`@GeneratedValue`**: Definește **`id_client`** ca cheia primară, care se generează automat cu strategie **`GenerationType.IDENTITY`** (valoare auto-increment în baza de date).
4. **`@Column`**: Folosit pentru a specifica proprietăți ale coloanelor bazei de date (de exemplu, lungimea, unicitatea, permisiunea `nullable` etc.).
5. **`@Data,@NoArgsCOnstructor, @AllArgsConstructor`**:Permit folosirea functiilor asociate librariei lombok pentru simplificarea codului
6. **Atributele clasei**: Contine atributele pentru **nume**, **email**, **telefon**,

7. Vom folosi getteri, setteri si constructorii din biblioteca lombok

Clasa **`Clienti`** reprezintă un obiect JPA care va fi salvat și gestionat în baza de date.

```
package com.example.manager_pro.Repository;

import com.example.manager_pro.clienti;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface clientiRepository extends JpaRepository<clienti, Integer> {

}
```

Aceasta este interfața clientitRepository, care extinde JpaRepository pentru a oferi operații CRUD (Create, Read, Update, Delete) pentru entitatea clienti.

JpaRepository<clienti, Integer>: Extinde funcționalitatea JpaRepository, care este un repository generic ce oferă metode predefinite pentru gestionarea entităților. În acest caz, clienti este tipul entității și Intteger este tipul cheii primare. Această interfață nu necesită implementări suplimentare.

```
package com.example.manager_pro.Controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.PathVariable;

import lombok.RequiredArgsConstructor;

import com.example.manager_pro.clienti;
import com.example.manager_pro.proiecte;
import com.example.manager_pro.clienti_proiecte;

import com.example.manager_pro.Repository.clientiRepository;
import com.example.manager_pro.Repository.proiecteRepository;
import com.example.manager_pro.Repository.clienti_proiecteRepository;
@Controller
```

```
@RequestMapping("/view")
@RequiredArgsConstructor

public class pageController {
    private final clientiRepository clientiRepo;
    private final proiecteRepository proiecteRepo;
    private final clienti_proiecteRepository clienti_proiecteRepo;

    @GetMapping("/dashboard")
    public String dashboard(){
        return "index";
    }
    //Clienti
    @GetMapping("/clienti")
    public String listClients(Model model) {
        model.addAttribute("lista", clientiRepo.findAll());
        return "clienti-list";
    }

    @GetMapping("/clienti/new")
    public String newClient(Model model) {
        model.addAttribute("obj", new clienti());
        return "clienti-form";
    }
    @GetMapping("/clienti/update/{id}")
    public String editClient(@PathVariable Integer id, Model model) {
        model.addAttribute("obj", clientiRepo.findById(id).orElseThrow());
        return "clienti-form";
    }

    @PostMapping("/clienti/save")
    public String saveClient(@ModelAttribute("obj") clienti c) {
        clientiRepo.save(c);
        return "redirect:/view/clienti";
    }

    @GetMapping("/clienti/delete/{id}")
    public String deleteClient(@PathVariable Integer id) {
        clientiRepo.deleteById(id);
        return "redirect:/view/clienti";
```

```
}

//Proiecte
@GetMapping("/proiecte")
public String listProjects(Model model) {
    model.addAttribute("lista", proiecteRepo.findAll());
    return "proiecte-list";
}

@GetMapping("/proiecte/new")
public String newProject(Model model) {
    model.addAttribute("obj", new proiecte());
    return "proiecte-form";
}
@GetMapping("/proiecte/update/{id}")
public String editProject(@PathVariable Integer id, Model model) {
    model.addAttribute("obj", proiecteRepo.findById(id).orElseThrow());
    return "proiecte-form";
}

@PostMapping("/proiecte/save")
public String saveProject(@ModelAttribute("obj") proiecte p) {
    proiecteRepo.save(p);
    return "redirect:/view/proiecte";
}

@GetMapping("/proiecte/delete/{id}")
public String deleteProject(@PathVariable Integer id) {
    proiecteRepo.deleteById(id);
    return "redirect:/view/proiecte";
}

//Clienti_Proiecte
@GetMapping("/clienti_proiecte")
public String listAssignments(Model model) {
    model.addAttribute("lista", clienti_proiecteRepo.findAll());
    return "clienti_proiecte-list";
}

@GetMapping("/clienti_proiecte/new")
```

```
    public String newAssignment(Model model) {
        model.addAttribute("obj", new clienti_proiecte());
        model.addAttribute("allClients", clientiRepo.findAll());
        model.addAttribute("allProjects", proiecteRepo.findAll());
        return "clienti_proiecte-form";
    }
    @GetMapping("/clienti_proiecte/update/{id}")
    public String editAssignment(@PathVariable Integer id, Model model) {
        // Find the assignment using id_rol
        clienti_proiecte assignment = clienti_proiecteRepo.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Invalid ID:"
+ id));

        // Add the found object to the model
        model.addAttribute("obj", assignment);

        // IMPORTANT: Repopulate dropdown lists for the form
        model.addAttribute("allClients", clientiRepo.findAll());
        model.addAttribute("allProjects", proiecteRepo.findAll());

        return "clienti_proiecte-form";
    }

    @PostMapping("/clienti_proiecte/save")
    public String saveAssignment(@ModelAttribute("obj") clienti_proiecte cp) {
        clienti_proiecteRepo.save(cp);
        return "redirect:/view/clienti_proiecte";
    }

    @GetMapping("/clienti_proiecte/delete/{id}")
    public String deleteAssignments(@PathVariable Integer id) {
        clienti_proiecteRepo.deleteById(id);
        return "redirect:/view/clienti_proiecte";
    }

}
```

Clasa **pageController** care centralizeazatoate metodele folosite in timpul interacriunii cu aplicatia, proceseaza si redirectioneaza corect fiecare cerere din GUI.

Fiecare entitate a aplicatiei este manageriata folosind doua template-uri html, nume-form si num-list, nume-form pentru gestionarea inserarii in baza de date, iar nume-list pentru afisare, datacenter gestioneaza editarea, afisarea si stergerea din baza de date, idex este front-pageul aplicatiei.

1. **Navigatie**

- Se utilizează **Bootstrap** pentru crearea mai rapida a interfetelor
- Link-urile sunt construite folosind sintaxa **Thymeleaf** (`th:href="@{...}"`), care este folosită pentru a rezolva dinamically URL-urile.

2. **Adaugarea clientilor(clienti-form)**

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
   <title>Manager Pro | New Client</title>
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-5">
   <div class="row justify-content-center">
       <div class="col-md-6">
           <div class="card shadow-sm border-0 p-4">
               <h2 class="fw-bold text-primary mb-4">Add New Client</h2>

                  <form th:action="@{/view/clienti/save}" th:object="${obj}"
method="post">
                    <input type="hidden" th:field="*{id_client}" />

                    <div class="mb-3">
                        <label class="form-label fw-bold">Full Name</label>
                                                <input type="text" th:field="*{nume}"
class="form-control" placeholder="Enter name" required>
                    </div>

                    <div class="mb-3">
                                                <label class="form-label fw-bold">Email
Address</label>
                                                <input type="email" th:field="*{email}"
class="form-control" placeholder="name@company.com" required>
                    </div>

                    <div class="mb-3">
                                                <label class="form-label fw-bold">Phone
Number</label>
                                               <input type="text" th:field="*{telefon}"
class="form-control" placeholder="07xx xxx xxx" required>
                    </div>

                    <div class="d-flex gap-2">
                                            <button type="submit" class="btn btn-primary
px-4">Save Record</button>
```

```
                                    <a th:href="@{/view/clienti}" class="btn
btn-outline-secondary">Cancel</a>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>
</body>
</html>
```

- **Buton de salvare și cancel**, pentru salvarea sau renuntarea la datele introduse
- Dupa salvare se redirecteaza catre afisarea tabelei **clienti**

3. Afisarea si alte operatii pe clienti angajat(clienti-list.html)

```
<!DOCTYPE html>

<html lang="en" xmlns:th="http://www.thymeleaf.org">

<head>

   <title>Client List</title>

                                                                                <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">

                                            <link              rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.0/font/bootstrap-ic
ons.css">

</head>

<body class="bg-light">

<div class="container mt-5">

   <div class="card shadow-sm border-0 p-4">

       <div class="d-flex justify-content-between align-items-center mb-4">

                           <h2 class="fw-bold text-primary"><i class="bi
bi-people-fill"></i> Client Database</h2>

           <div>

                                <a th:href="@{/view/dashboard}" class="btn
btn-outline-secondary me-2">Back</a>
```

```
                <a th:href="@{/view/clienti/new}" class="btn btn-primary">+
Add New</a>

            </div>

        </div>



        <table class="table table-hover align-middle">

            <thead class="table-light">

            <tr>

                <th>ID</th>

                <th>Name</th>

                <th>Email</th>

                <th>Phone</th>

                <th class="text-center">Actions</th>

            </tr>

            </thead>

            <tbody>

            <tr th:each="c : ${lista}">

                <td th:text="${c.id_client}" class="fw-bold"></td>

                <td th:text="${c.nume}"></td>

                <td th:text="${c.email}"></td>

                <td th:text="${c.telefon}"></td>

                <td class="text-center">

                                                                                <a
th:href="@{/view/clienti/update/{id}(id=${c.id_client})}" class="btn btn-sm
btn-outline-primary">

                        <i class="bi bi-pencil"></i> Edit

                    </a>

                                                                                <a
th:href="@{/view/clienti/delete/{id}(id=${c.id_client})}"

                        class="btn btn-sm btn-outline-danger"
```

```
                        onclick="return confirm('Are you sure you want to
delete this client?')">

                        <i class="bi bi-trash"></i>

                    </a>

                </td>

            </tr>

            </tbody>

        </table>

    </div>

</div>

</body>

</html>
```

- Este folosit **Thymeleaf** pentru iterarea si accesul la coloanele tabelei clienti`th:each="c : ${lista}"` sau `th:text="${c.id_client}" class="fw-bold"`
- Link-urile actiunilor respective sunt create ca anterior cu ajutorul **Thymeleaf**
- Sunt adaugate butoane pentru Edit, Delete, formularul de adaugare si intoarcerea catre actiuni anterioare

6. Concluzii:

Bazele de date sunt fundamentale pentru gestionarea și procesarea informațiilor în diverse domenii și industrii. Aplicația dezvoltată folosind tehnologia Spring, împreună cu SQL pentru gestionarea bazei de date și HTML/CSS pentru interfața utilizatorului, oferă o arhitectură modernă și eficientă, separând clar logica de back-end de partea de front-end. Această separare permite utilizatorilor să efectueze operațiuni esențiale asupra unei baze de date MySQL, cum ar fi vizualizarea, adăugarea, actualizarea și ștergerea datelor angajaților. Framework-ul Spring se dovedește a fi o alegere robustă pentru dezvoltarea aplicațiilor web, oferind un control mai bun asupra aplicației, scalabilitate și performanță ridicată. Designul modular al aplicației asigură o administrare mai eficientă a datelor și o experiență de utilizare intuitivă, contribuind astfel la optimizarea proceselor și la îmbunătățirea performanței generale a aplicației.

7. Bibliografie

[1] "MySQL Documentation," MySQL, Oracle. [Online]. Available: https://dev.mysql.com/doc/. [Accessed: 07-Jan-2026].

[2] "Spring Boot Documentation," Spring Boot, VMware/VMware Tanzu. [Online]. Available: https://docs.spring.io/spring-boot/index.html. [Accessed: 07-Jan-2026]

[3] "HTML — MDN Web Docs," *developer.mozilla.org*, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. [Accessed: 07-Jan-2026]

[4] "CSS — MDN Web Docs," *developer.mozilla.org*, Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS. [Accessed: 07-Jan-2026].

[5] "Spring Boot – Database Integration (JPA, Hibernate, MySQL, H2)," *GeeksforGeeks*. [Online]. Available: https://www.geeksforgeeks.org/advance-java/spring-boot-database-integration-jpa-hibernate-mysql-h2/. [Accessed: 07-Jan-2026]

[6] J. Chan, *Learn Java in One Day and Learn It Well: Java for Beginners with Hands-On Project*, Independently published, 2016. ISBN: 9781790789870

[7] "Introduction to Bootstrap 4.1," *getbootstrap.com*. [Online]. Available: https://getbootstrap.com/docs/4.1/getting-started/introduction/. [Accessed: 07-Jan-2026]